AlphaLens: System Blueprint & Contribution Matrix

Operationalizing Autonomous Multi-Agent Causal Alpha Discovery via LangGraph

Project Source: ELITE_NEW (1).pdf (IIT Dharwad)
Original Authors: Divyansh Prakash & Harsh Mishra
Release Date: May 6, 2026
Document Class: Technical Integration Specification
Target Framework: LangGraph / State Machine Orchestration
AUM Constraint: Capacity bounded at $500M US Equities

1. Introduction to the Agentic System Architecture

In traditional computing frameworks, code executes along a predefined, linear path dictated by hard-coded conditions. AlphaLens marks a complete structural shift by organizing around an **Agentic Ecosystem**. The platform consists of independent, software-driven "workers" (Agents) assigned distinct roles, explicit objective boundaries, personal toolkits, and dynamic memories. Rather than following static operational sequences, these agents use underlying Large Language Models (LLMs) and probabilistic frameworks to evaluate, branch, and optimize pathways autonomously.

The orchestrator utilizes **LangGraph** to formalize this ecosystem as a Directed Acyclic Graph with explicit state properties. The network is fundamentally event-driven; individual agents pass typed message payloads over a centralized state log, allowing the system to continuously audit and re-route logic loops based on statistical significance thresholds.

2. Joint Engineering Ownership & Equal-Worth Paradigm

Developing a fully autonomous quantitative platform requires two separate but equally vital technical domains: Algorithmic Intelligence and Scalable Orchestration. Neither discipline can function in isolation. Without a high-fidelity intelligence framework, the pipeline automates invalid, un-orthogonalized noise. Conversely, without production-grade systems architecture, advanced analytical scripts remain localized, unable to stream data at pace or serialize memory. The work is explicitly divided into two equivalent engineering roles.

Person A: Quant & AI Specialist (The Intelligence Architect)	Person B: Systems & Data Platform Engineer (The Systems Architect)
Core Mandate: Engineering agent reasoning boundaries, mathematical toolkits, deep learning feature backbones, and causal inference gating models.	**Core Mandate:** Managing multi-agent workflow runtime states, time-series streaming storage networks, execution safety rails, and computing layers.
Career & Asset Value: Masters domain-specific AI modeling. Translates raw theoretical text and academic parameters into algorithmic features that maintain an alpha edge.	**Career & Asset Value:** Masters high-distributed agentic graph architecture. Standardizes agent memory persistence and guards against latency or state drift.

> **Systemic Dependency Interloop**
>
> If Person A's mathematical logic fails, the system executes beautifully automated, high-throughput financial errors. If Person B's platform framework fails, the mathematical models fail to ingest point-in-time data or synchronize, preventing execution entirely. The roles form an equal partnership.

3. Comprehensive Engineering Task Matrix

PERSON A: CORE ENGINEERING IMPLEMENTATIONS

1. **Agent Personality, System Prompt Engineering, and RAG Extraction:** Construct contextual boundaries for the `Literature Agent`. Program the paragraph-level semantic processing pipelines (FAISS/Chroma flat-L2 indices) using text-embedding-3-large models to extract clean, JSON-formatted structural hypotheses.
2. **Mathematical Feature & Predictive Screening Toolkits:** Build the feature factory computing 312 base parameters. Code the standalone statistical tools for the `Signal Generation Agent` to parse Information Coefficients (IC via Spearman rank correlation) and Information Ratios ($ICIR$).
3. **Probabilistic Deep Learning & GNN Backbones:** Build and train the primary multi-horizon predictive networks (Temporal Fusion Transformer, N-BEATS trend-seasonality layers, and PatchTST patch-boundary modules). Build the dynamic correlation Graph Attention Network (GAT) to process cross-border asset relationships.
4. **Causal Verification Framework:** Implement the structural validation scripts for the `Causal Validation Agent`. Write the PC-algorithm and Fast Causal Inference (FCI) models to map Directed Acyclic Graphs (DAGs) and build the 5-fold cross-fitted Double/Debiased Machine Learning (DML) estimator for Average Treatment Effect (ATE) extraction.
5. **Risk-Adjusted Portfolio Allocation:** Build the Mean-CVX portfolio optimizer using CVXPY with a CLARABEL backend, mapping the Rockafellar-Uryasev linearization alongside structural Black-Litterman matrix balances.

PERSON B: CORE ENGINEERING IMPLEMENTATIONS

1. **Time-Series & Relational Storage Layer:** Deploy and maintain the main database cluster using TimescaleDB extensions on PostgreSQL. Formulate hypertable indexing rules (7-day intervals for sub-daily ticks, 30-day partitions for daily records) to secure $O(\log n)$ performance over long range queries.
2. **High-Throughput Streaming & Cache Caching Management:** Connect Apache Kafka brokers to ingestion endpoints to manage streaming real-time data cleanly. Configure Redis caching nodes to provide sub-millisecond layer delivery for active cross-sectional factor matrices.
3. **State Machine Graphs & Orchestration Protocol:** Build the structural graph network in LangGraph. Define the central state schema dictionary, assign execution loops to core nodes, and implement serialized Protocol Buffer schemas (M) to manage all cross-agent messages.
4. **Simulation Framework & Gating Parameters:** Code the out-of-sample simulation framework using vectorized engines (vectorbt/Zipline-reloaded). Construct automated guards for execution slippage, commissions, and Kyle's lambda transaction costs while enforcing point-in-time temporal data separation.
5. **Kubernetes MLOps & Operational Dashboards:** Dockerize all computing nodes and configure auto-scaling group deployment charts inside an AWS EKS cluster (GPU nodes for learning phases, CPU groups for baseline parsing). Connect MLflow experiment tracking databases and build the Streamlit diagnostic web panel.

4. Hard Interface Contracts Between Both Architects

To allow independent development without breaking cross-module connectivity, both engineers must adhere to three rigid technical boundaries:

CONTRACT 1: THE LANGGRAPH SHARED STATE SCHEMA

Person B provides a locked, type-safe dictionary schema handled via PostgreSQL. Person A's algorithmic agent scripts must consume this state block as raw input and output updates using matching datatypes. For example, the `Literature Agent` output must validate against this exact format:

- `hypothesis_id` (String, checked for unique index keys)
- `predictor_variable` / `target_asset_class` (Strings matching global feature names)
- `predicted_direction` (Enum: `positive`, `negative`)
- `confidence` (Float bounded between `0.00` and `1.00`)

CONTRACT 2: THE STATISTICAL GATING THRESHOLDS

The automated workflow shifts execution lanes dynamically based on statistical markers. Person A's validation algorithms must populate two variables in the graph's memory block: `p_value` (Float) and `ate_magnitude` (Float). Person B's LangGraph routing architecture implements this exact condition:

```
if state["p_value"] < 0.05 and state["sharpe_ratio"] >= 1.0:
    return "ROUTE_TO_PORTFOLIO_CONSTRUCTION"
else:
    return "ROUTE_TO_REJECTION_AND_REFINEMENT_LOOP"
```

CONTRACT 3: LOOK-AHEAD BIAS ISOLATION (TEMPORAL API)

To prevent data-leakage and look-ahead bias during deep learning training, Person A's training scripts are blocked from making direct, unmonitored SQL data commands. Person B exposes an isolated API proxy layer. This proxy acts as a filter, validating that queries with a timestamp parameter t never return observations, corporate revisions, or pricing history beyond that exact milestone.

5. Project Launch Strategy & First-Month Milestones

The development pipeline starts with a coordinated phase to align infrastructure with modeling tools before deeper modules are built out.

- **Week 1 (Interface Alignment):** Person B deploys the TimescaleDB foundation and initializes a hollow 5-node LangGraph loop. Person A scripts the standalone Python tools for Spearman rank IC calculations and builds the baseline System Prompts. Both engineers must meet to cross-verify the Protobuf messaging structures.
- **Week 2 (Data Ingestion & Matrix Connect):** Person B maps the data streaming loaders to populate base tables. Person A completes the initial 50 core feature algorithms and loops them through Person B's Temporal API endpoints to verify look-ahead protection.
- **Week 3 (Intelligence Integration):** Person B configures the LangGraph memory engines (Working, Episodic, and Semantic stores). Person A integrates the base PC-algorithm causal discovery code inside the `Causal Validation Agent` node.
- **Week 4 (End-to-End System Integration Test):** Run a complete system integration test: Lit Agent extracts a hypothesis $ ightarrow$ Signal Agent computes the feature matrix $ ightarrow$ Causal Agent returns a mock p-value $ ightarrow$ LangGraph successfully routes or rejects the process.